



成绩 _____

北京航空航天大学
BEIHANG UNIVERSITY

微机原理及接口技术 实验报告

院（系）名称 自动化科学与电气工程学院

专业名称 自动化

学生学号 22375413

学生姓名 夏思博

指导教师 林新

2023 年 11 月

实验一 并行接口实验

实验时间 第十三周周四 2-5 节

同组同学 王梓枫

一、实验目的

- 1、学习和理解接口芯片的地址译码电路,掌握 IO 译码电路设计;
- 2、熟练掌握并行接口芯片 8255A 的基本原理、初始化设置、工作方式 0 的应用;
- 3、学习和掌握数码管动态显示数字的原理;
- 4、对比 EDA 软件设计与虚拟仿真和实验箱实物实验的异同

二、实验原理

1. 说明实验中涉及到的相关理论、概念、算法或者原理

电路原理图如图 1 所示。地址译码器输出 (288H~28FH) 片选信号, 连接 8255A 的 CS 引脚。

8255A 的 PA0~PA6 分别连接数码管的 LED 灯 a~g 的阳极; PC0 连接 0#数码管的 LED 灯 a~g 的阴极, PC1 连接 1#数码管的 LED 灯 a~g 的阴极。

实验箱上的七段数码管为共阴型, 阴极输入端加低电平, 选中的数码管亮。

8255A 的 PA 端口引脚、PC 端口的 PC0、PC1 即可控制数码管中 LED 灯的亮和灭。例如, 设 PA1、PA2 输出高电平(逻辑 1), 其它 PA 端口引脚输出低电平(逻辑 0), 若 PC0 输出低电平(逻辑 0), 则 0#数码管的 LED 灯 b、c 点亮, 显示数字“1”; 若此时 PC1 输出高电平(逻辑 1), 则 1#数码管不显示。

设 PA0、PA1、PA2 输出高电平(逻辑 1), 其它 PA 端口引脚输出低电平(逻辑 0), 若 PC1 输出低电平(逻辑 0), 则 1#数码管的 LED 灯 a、b、c 点亮, 显示数字“7”; 若此时 PC0 输出高电平(逻辑 1), 则 0#数码管不显示。

当 PA 端口输出两个不同的数值编码, 同时 PC0、PC1 交替输出 0 和 1, 则两

个数码管交替显示不同的数字。当交替显示的频率非常快的时候，仿佛两个数码管同时显示两个数字。

2. 附上电路原理图、流程图

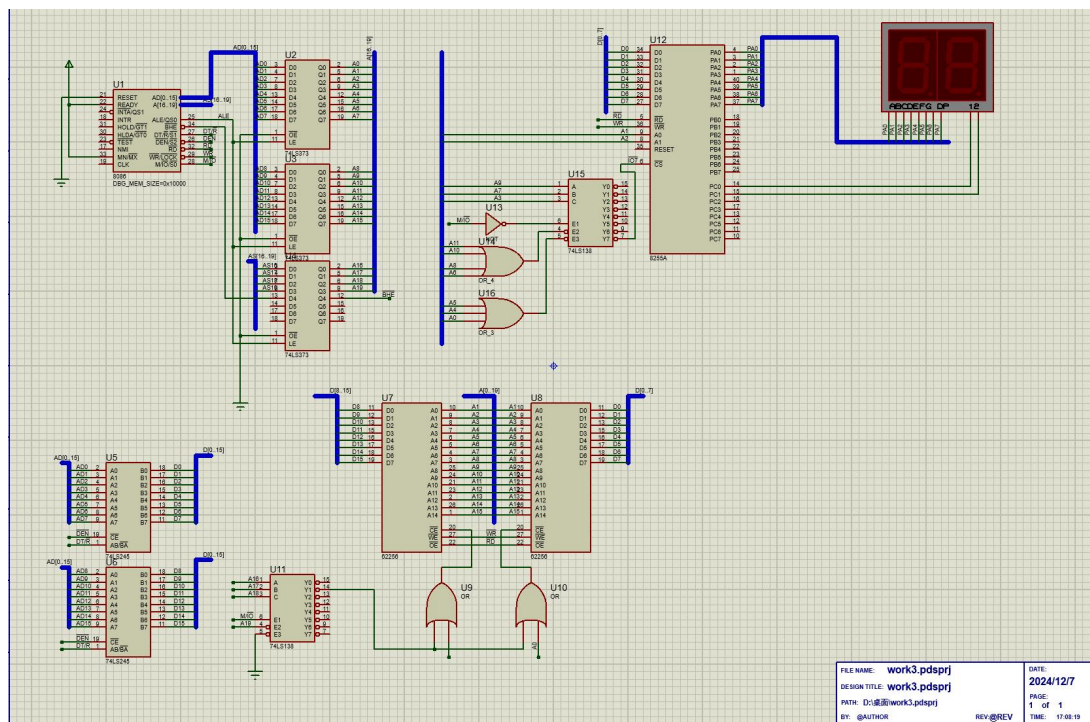


图 1 实验内容 2（实验六）电路仿真原理图

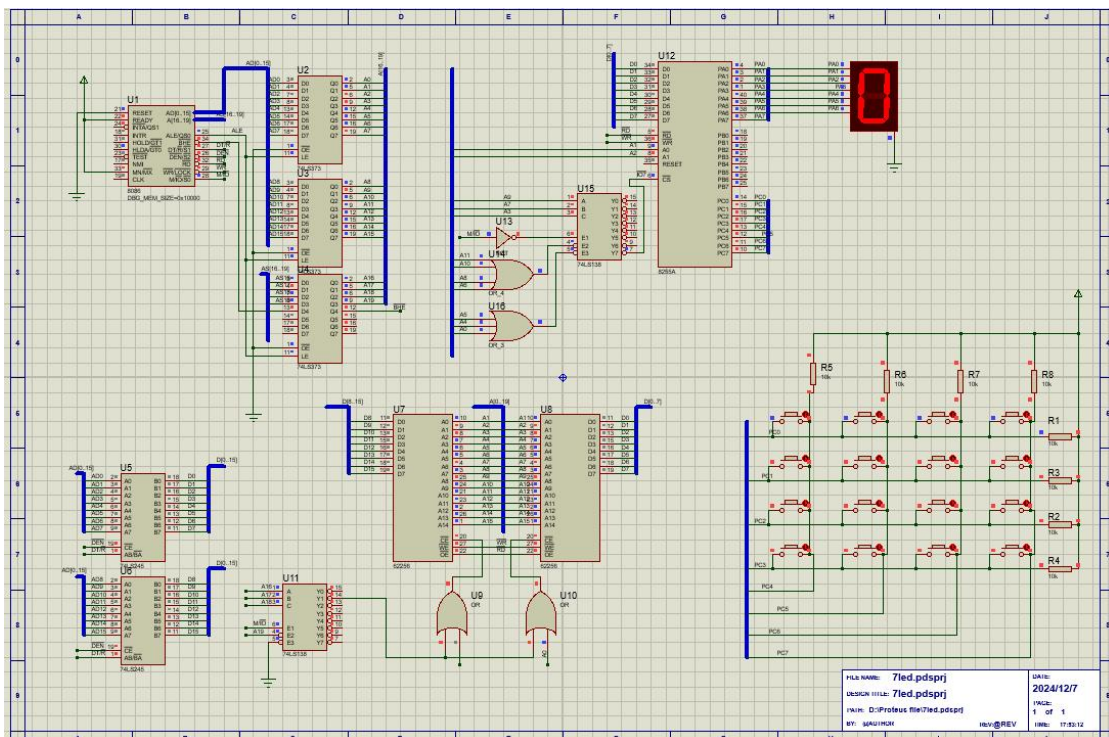


图 2 4*4 数字键盘电路原理图

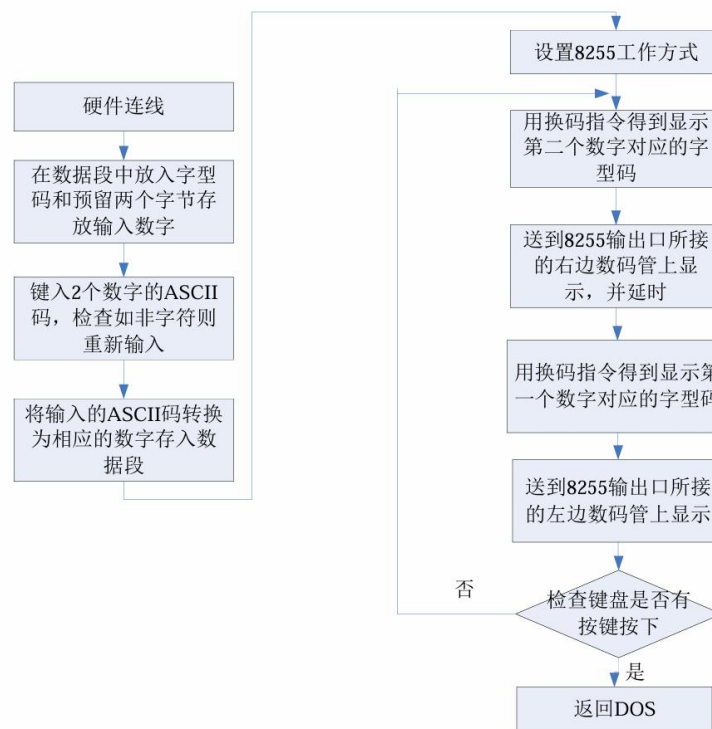


图3 实验流程图

三、实验内容与实验过程描述

一：实验箱搭建电路显示数字字符

1、连接地址译码器输出与 8255 的片选信号，以及 8255 的与数码管的连线，如图 2。按图 2 连接电路，将 8255 的 A 口 PA0~PA6 分别与七段数码管的段码驱动输入端 a~g 相连(使用排线)，位码驱动输入端 S1, S0 接 8255C 口的 PC1, PC0。片选信号 CS 接地址译码器输出的 288H~28FH 端。

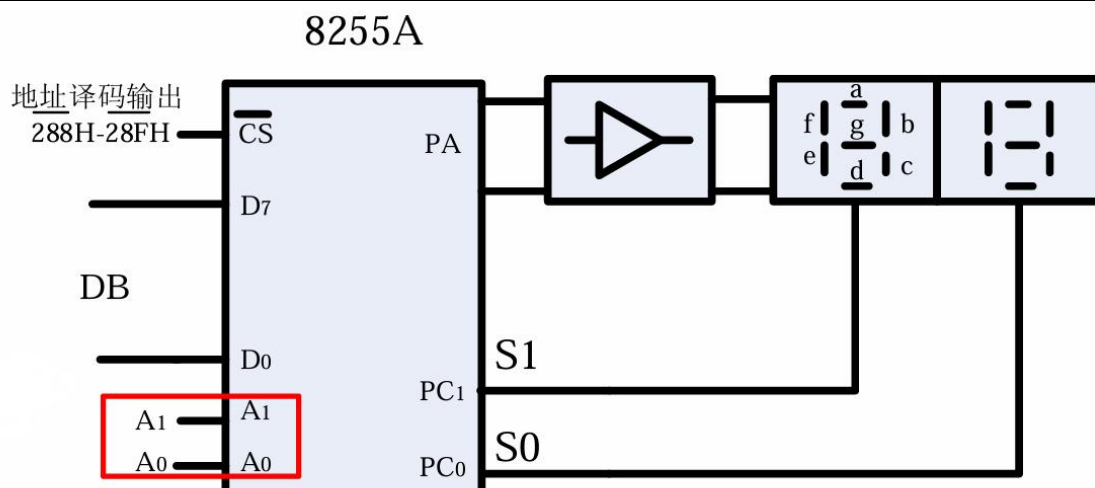


图 4 8255A 简化电路原理图

2、连接实验箱与计算机，并开启计算机

3、打开运行 TPC-ZK-II 集成开发环境；在 TPC-ZK-II 集成开发环境运行界面下建立或打开已编写好汇编语言源程序文件，对程序进行编译、连接、运行。

4、运行程序，从键盘输入两位数字，观察数码管上的数据显示。

二：在 proteus 上进行仿真：

基本和上述一致，只在设计地址译码电路时有不同，实验箱上的端口地址是连续的与实际 8086 不符，所以地址总线的 A2 和 A1 分别连接到 8255A 的 A1 和 A0，并设计译码电路来控制端口 A 的地址为 288H、接着搭建 4*4 键盘按钮，换成一个数码管。

四、实验结果分析与实验结论

1. 对实验所得结果进行分析处理，总结实验结论。

(1) 实验内容 1-搭建实验箱显示数字字符

首先确定 8255A 的工作方式，输出控制字给控制端口，控制端口 A 输出，端口 C 低四位输出，其次利用 8 号功能从键盘输入字符（如果输入不为 0-9 的数字字符重新输入），输入两个数字字符，同时清空高四位使其转为 BCD 码，并将其存入数据段。

其次从端口输出输入数字对应的七段数码管对应的字型代码，并控制端口 C 输出 01B 使只有第一个数码管亮，之后在利用两个循环起到延迟作用，在控制端口 C 输出 11B

清空数码管。在输出第二个数字，过程和第一个输出数字过程类似。利用 6 号功能，若没有键盘输入则重复显示之前输入的两个数字，若有键盘输入则在重新输入两个数字字符显示。

最终可以从键盘输入两个 0-9 的数字字符并交替显示在数码管上，并可以重复输入数字来显示。

(2) 实验内容 2

和实验内容 1 几乎一致，控制两个数字管分别显示 1 和 0 交替闪烁。

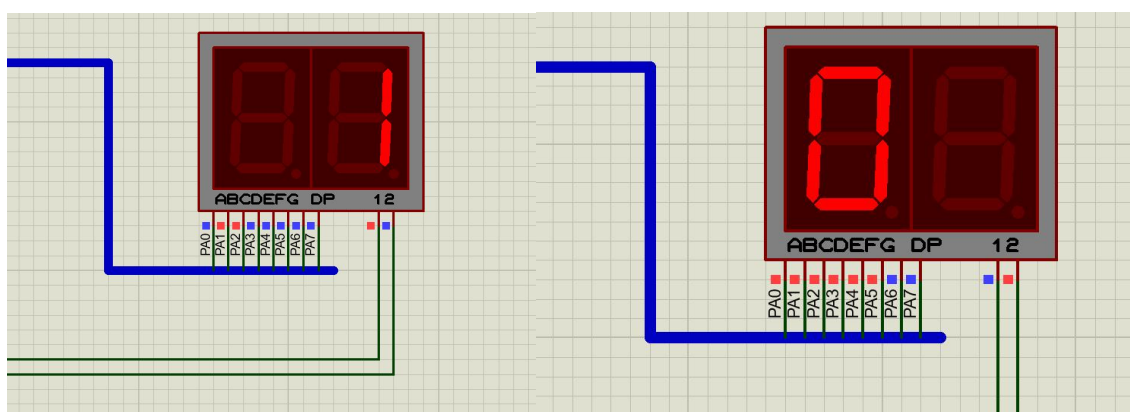


图 5 实验内容 2 效果图

(2) 实验内容 3

16 个按钮按下可以在数码管上显示出 16 进制数字 1~F

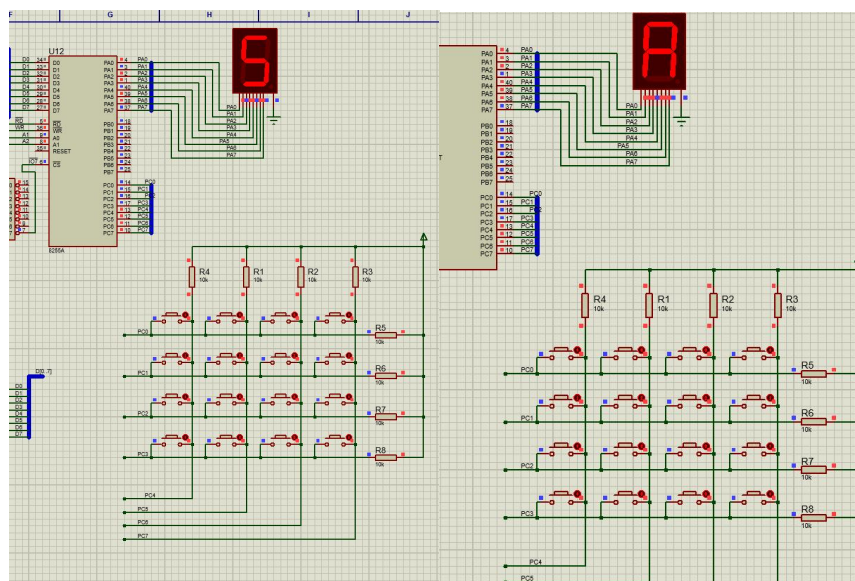


图 6 4*4 键盘输入 5 和 A 样例图

2. 实验过程中遇到的问题分析总结。

实验内容 1 实物接线中和实验内容 2 模拟的地址引脚接线不一致，仿真中 A0、A1

接入的是 A2、A1，因此地址也不一样。

五、收获、体会及建议

1. 总结实验收获与个人体会；欢迎同学提出改善实验课程的建议。

在实验中我学会了通过七段码用 8255A 输出 10 进制数字和 16 进制数字，并且学会了编写键入子程序和延迟子程序，让晶体管成功显示出数字。并且，我了解了仅仅使用 TPC 环境没有硬件无法运行的晶体管显示。在建议上，我建议增加一些实验课时，让大家先熟悉一下 8086 系统以及汇编，然后稍微减少一下每节课的时长，这样能做更多更有趣的内容了。

附录 1

1、预习思考题及实验验证

1. 计算采用共阴极七段数码管显示的 0 到 9 的字形码(十六进制数表示);

显示字形	g	f	e	d	c	b	a	段码
0	0	1	1	1	1	1	1	3FH
1	0	0	0	0	1	1	0	06H
2	1	0	1	1	0	1	1	5BH
3	1	0	0	1	1	1	1	4FH
4	1	1	0	0	1	1	0	66H
5	1	1	0	1	1	0	1	6DH
6	1	1	1	1	1	0	1	7DH
7	0	0	0	0	1	1	1	07H
8	1	1	1	1	1	1	1	7FH
9	1	1	0	1	1	1	1	6FH
A	1	1	1	0	1	1	1	77H
B	1	1	1	1	1	0	0	7CH
C	0	1	1	1	0	0	1	39H
D	1	0	1	1	1	1	0	5EH
E	1	1	1	1	0	0	1	79H
F	1	1	1	0	0	0	1	71H

图 7 共阴极七段码

2. 编写键盘输入数字程序/子程序，要求在输入过程中检查是否是数字键，不是则不显示键入数字并重新输入;

KEYIN PROC NEAR ; 从键盘输入数字，若不为数字则重新输入

BEG:

MOV AH, 08H ;8号功能键入

INT 21H

CMP AL, 30H

JB BEG ;小于0跳转

CMP AL, 39H

JA BEG ;大于9跳转

MOV DL, AL ;将键入的值放入AL寄存器

RET

KEYIN ENDP

3. 编写显示器显示键入数字程序/子程序，要求当输入正确的数字后能显示键入数字（即比第2题多了显示功能）；

KEYIN PROC NEAR ; 从键盘输入数字并显示，若不为数字则重新输入

BEG:

```
MOV AH, 08H ;8号功能键入
INT 21H
CMP AL, 30H
JB BEG ;小于0跳转
CMP AL, 39H
JA BEG ;大于9跳转
MOV AH, 02H ;2号功能显示
PUSH AX
MOV DL, AL
INT 21H
POP AX
RET
```

KEYIN ENDP

4. 编写延时程序/子程序，适当设置延时时间，保证两位数码管正确、稳定可靠地显示键入的两位数字。

DELAY PROC NEAR ;DELAY 起到延迟的作用

MOV BX, 64H ;外循环100

NEXT1:

MOV CX, 3E8H ;内循环1000

NEXT2:

```
LOOP NEXT2
DEC BX
JNZ NEXT1
RET
```

DELAY ENDP

2、实验后思考题

1. 有 01 号、06 号、07 号、08 号、0AH 号功能可以输入字符，01 号是输入一个字符并回显，输入字符存入 AL，06 号当 DL=0FFH 时可以输入字符，判断是否有键盘按下，若无按键按下则 ZF=1，JZ 跳转，不等待键入。07 号键盘输入不回显、不检查回车键，08 号输入不回显、检查回车。0AH 功能从键盘接收字符，并存放至内存缓冲区。在使用 0AH 号功能调用时，执行前先定义一个输入缓冲区，缓冲区内第一个字节定义

为允许最多输入的字符个数，字符个数应包括回车符 0DH 在内，不能为"0"值。第二个字节保留，在执行程序完毕存入输入的实际字符个数。从第三个字节开始存入从键盘上接收字符的 ASCII 码。

2. 实验中数码管为共阴极数码管，从 8255A 端口 A 链接数码管控制输出，端口 C 对应连接阴极，当 A 端口对应输出高电平，对应 C 端口输出低电平，则对应晶体管会亮。若为共阳极，则当 C 端口输出高电平，对应 A 端口输出低电平时才会使晶体管亮。

3. 两个数码管交替显示不同的数字，当交替显示的频率非常快的时候，仿佛两个数码管同时显示两个数字。延时时间不能太长，否则两个数字显示时间间隔太长，人眼会观察出交替显示。也不能太短，否则显示时间太短，不太能看出来显示的数字。

4. 可以，只需改变控制字，使 B 端口工作方式 0 并输出，并将要输出的数据从 B 端口输出就行。

附录 2：实验源程序

1、实物实验的源程序如下：

```
DATA SEGMENT
    DT1 DB ?
    DT2 DB ?
    LIST DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH; 0-9 字型码
DATA ENDS

STKS SEGMENT STACK
    DW 100 DUP(0)
STKS ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA, SS:STKS
MAIN PROC FAR
    ; 初始化及数据段、堆栈段观察记录
    MOV AX, STKS
    MOV SS, AX
    MOV SP, 100*2

    MOV AX, DATA
```

```
MOV DS, AX
```

```
MOV AL, 8AH ;确定8255A工作方式
```

```
MOV DX, 28BH ;8255A控制端口
```

```
OUT DX, AL
```

```
AA:
```

```
CALL KEYIN ;键盘输入两个数字，分别放入DT1与DT2中
```

```
AND AL, 0FH ;ASCII码转非压缩BCD码
```

```
MOV DT1, AL
```

```
CALL KEYIN
```

```
AND AL, 0FH
```

```
MOV DT2, AL
```

```
MOV BX, 0
```

```
AGN:
```

```
MOV BL, DT1
```

```
MOV AL, LIST[BX]
```

```
MOV DX, 288H ;A端口输出DT1的七段码
```

```
OUT DX, AL
```

```
MOV DX, 28AH ;C端口控制第一个显示管亮，第二个显示管灭
```

```
MOV AL, 01B
```

```
OUT DX, AL
```

```
CALL DELAY ;延迟
```

```
MOV DX, 28AH ;让两个都灭
```

```
MOV AL, 0
```

```
OUT DX, AL
```

```
MOV BL, DT2
```

```
MOV AL, LIST[BX]
```

```
MOV DX, 288H ;A端口输出DT2的七段码
```

```
OUT DX, AL
```

```
MOV DX, 28AH ;C端口控制第二个显示管亮
```

```
MOV AL, 10B
```

```
OUT DX, AL
```

```
CALL DELAY ; 延迟
```

```
MOV DX, 28AH ;让两个都灭
```

```
MOV AL, 0
```

```
OUT DX, AL
```

```
MOV AH, 06H ;6号功能检测按键是否按下
```

```
MOV DL, 0FFH
```

```
INT 21H
```

```
JNZ AGN          ;若无按键按下则跳转AGN继续显示
CMP AL, 27        ;检查是否按下ESC
JE EXIT          ;如果是ESC则退出
JMP AA           ;否则回到开头重新读取数字并显示

EXIT:
    MOV AH, 4CH
    INT 21H

MAIN ENDP

DELAY PROC NEAR    ;DELAY 起到延迟的作用
    MOV BX, 64H    ;外循环100
NEXT1:
    MOV CX, 3E8H   ;内循环1000
NEXT2:
    LOOP NEXT2
    DEC BX
    JNZ NEXT1
    RET
DELAY ENDP

KEYIN PROC NEAR    ; 从键盘输入数字并显示，若不为数字则重新输入
BEG:
    MOV AH, 08H    ;8号功能键入
    INT 21H
    CMP AL, 30H
    JB BEG         ;小于0跳转
    CMP AL, 39H
    JA BEG         ;大于9跳转
    MOV AH, 02H    ;2号功能显示
    PUSH AX
    MOV DL, AL
    INT 21H
    POP AX
    RET
KEYIN ENDP

CODE ENDS
END MAIN
```

2、proteus 仿真实验（实验六）源程序做如下改动：

```
MOV AL, 3FH      ;数字管1显示0
MOV DX, 288H
OUT DX, AL
MOV DX, 28CH
MOV AL, 10B
OUT DX, AL
CALL DLAY ;DELAY起到延迟的作用
```

```
MOV DX, 28CH
MOV AL, 00B
OUT DX, AL
```

```
MOV AL, 06H      ;数字管0显示1
MOV DX, 288H
OUT DX, AL
MOV DX, 28CH
MOV AL, 01B
OUT DX, AL
CALL DLAY
```

2、4*4 键盘输入源程序如下：

```
DATAS SEGMENT
MY8255_A EQU 288H
MY8255_B EQU 28AH
MY8255_C EQU 28CH
MY8255_MODE EQU 28EH
LED DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H
    DB 7FH, 6FH, 77H, 7CH, 39H, 5EH, 79H, 71H, 00H
CURRENT_LED DB 10H
ROW_DISP DB 0FEH, 0FDH, 0FBH, 0F7H
DATAS ENDS
```

```
STACKS SEGMENT
;此处输入堆栈段代码
DW 50 DUP(?)
TOP LABEL BYTE
STACKS ENDS
```

```
CODES SEGMENT
ASSUME CS:CODES, DS:DATAS, SS:STACKS
START:
MOV AX, STACKS
```

```

MOV SS, AX
MOV SP, OFFSET TOP
MOV AX, DATAS
MOV DS, AX
;此处输入代码段代码
;写入控制字
MOV AL, 10001000B;A为输出信号, C的高位为输入信号列, 低位为输出信号行
MOV DX, MY8255_MODE
OUT DX, AL
CALL DISP
MOV BH, 00H;从第零行开始扫描
MOV BL, 00H
RESTART:
    CALL DISP
DETECT_ROW:
    MOV DX, MY8255_C
    MOV AL, ROW_DISP[BX]
    OUT DX, AL
    IN AL, DX
    AND AL, 0F0H
    CMP AL, 0F0H
    JNC NEXT_LINE;无输入直接转到下一行
    MOV CX, 5000H
    LOOP $
    IN AL, DX
    AND AL, 0F0H
    CMP AL, 0F0H
    JNC NEXT_LINE
    MOV DL, 0;用于记录该行中列的数据
    MOV CX, 4
FIND_LINE:
    SHL AL, 1
    JNC RECORD_LINE;无进位说明是列
    INC DL
    LOOP FIND_LINE
RECORD_LINE:
    MOV AL, BL;行的数据保存在BL中, 列的数据保存在DL中
    SHL AL, 1
    SHL AL, 1
    MOV DH, 03H
    SUB DH, DL
    ADD AL, DH
    MOV CURRENT_LED, AL
    CALL DISP

```

;记录之后继续扫描下一行

NEXT_LINE:

```
CALL DELAY
ADD BL, 01H
AND BL, 0FBH
JMP RESTART
```

DISP PROC NEAR;数码管显示部分

```
MOV BL, CURRENT_LED
MOV BH, 00H
MOV AL, LED[BX]
MOV DX, MY8255_A
OUT DX, AL
RET
```

DISP ENDP

DELAY PROC NEAR;软件延时程序

```
PUSH CX
PUSH AX
MOV CX, 011H
```

D1:

```
MOV AX, 00FFH
```

D2:

```
DEC AX
JNZ D2
LOOP D1
POP AX
POP CX
RET
```

DELAY ENDP

EXIT:

```
MOV AH, 4CH
INT 21H
```

CODES ENDS

```
END START
```

实验系统指导书中 14-17 页的实验预习题

课前预习 1:

```

Turbo Assembler Version 5.3      11-07-24 13:41:54      Page 1
ASMwork(1).asm

1 0000          DATA SEGMENT
2 0000 25      DT1 DB 37
3 0001 64      DT2 DB 64H
4 0002 ABCD    DT3 DW 0ABCDH
5 0004 8765    DT4 DW 8765H
6 0006 =00A7   DT5 EQU 0A7H
7          DATA ENDS
8
9 0000          STKS SEGMENT STACK
10 0000 64*(0000) DW 100 DUP(0)
11 00C8          STKS ENDS
12
13 0000          CODE SEGMENT
14          ASSUME CS:CODE, DS:DATA, SS:STKS
15 MAIN PROC FAR
16 ;初始化及数据段、堆栈段观察记录
17 0000 B8 0000s MOV AX, STKS ; (1)
18 0003 8E D0    MOV SS, AX
19 0005 BC 00C8  MOV SP, 100*2
20
21
22 0008 B8 0000s MOV AX, DATA
23 000B 8E D8    MOV DS, AX ; (2)
24
25 ;加减法、PSW 实验
26 000D A1 0002r MOV AX, DT3 ; (3)
27 0010 03 06 0004r ADD AX, DT4 ; (4)
28 0014 A3 0004r MOV DT4, AX ; (5)
29
30 0017 A0 0000r MOV AL, DT1
31 001A 8A 1E 0001r MOV BL, DT2
32 001E 2A C3    SUB AL, BL ; (6)
33 0020 A2 0000r MOV DT1, AL ; (7)
34
35 ;比较、条件转移、CALL 指令以及堆栈实验
36 0023 3B 06 0002r CMP AX, DT3 ; (8)
37 0027 77 04    JA NEXT1
38 0029 50      PUSH AX ; (9)
39 002A EB 05 90 JMP NEXT2
40 002D          NEXT1:
41 002D FF 36 0002r PUSH DT3 ; (10)
42 0031          NEXT2:
43 0031 5A      POP DX ; (11)
44 0032 E8 0004 CALL SPRC ; (12)
45
46 0035 B4 4C    MOV AH, 4CH
47 0037 CD 21    INT 21H ; (15)
48
49 0039          MAIN ENDP
50
51 0039          SPRC PROC NEAR
52 0039 B1 03    MOV CL, 3
53 003B D3 C2    ROL DX, CL ; (13)
54 003D C3      RET ; (14)
55 003E          SPRC ENDP
56 003E          CODE ENDS
57          END MAIN

```

可见标号（3）处的偏移地址为 000DH，机器码为 A1 0002r

标号（12）处的偏移地址为 0032H，机器码为 E8 00042

2.执行（1）之前的内部寄存器：

寄存器	地址	内容
EAX:	0x0000FE7	
ECX:	0x000E0000	
EDX:	0x00002002	
EBX:	0x0000FEA	
ESP:	0x00000C6	
EBP:	0x0000011E	
ESI:	0x00000100	
EDI:	0x0000024A	
EIP:	0x00000000	
EFLAGS:	0x00007202	
CS:	0x0FF4	
SS:	0x0FE7	
DS:	0x0F7C	
ES:	0x0F7C	
FS:	0x0000	
GS:	0x0000	

地址	内容
0001	DATA SEGMENT
0002	DT1 DB 37
0003	DT2 DB 64H
0004	DT3 DW 0ABCDH
0005	DT4 DW 8765H
0006	DT5 EQU 0A7H
0007	DATA ENDS
0008	
0009	STKS SEGMENT STACK
0010	DW 100 DUP(0)
0011	STKS ENDS
0012	
0013	CODE SEGMENT
0014	ASSUME CS:CODE, DS:DATA, SS:STKS
0015	MAIN PROC FAR
0016	:初始化及数据段、堆栈段观察记录
0017	MOV AX, STKS ;(1)
0018	MOV SS, AX
0019	MOV SP, 100*2
0020	
0021	
0022	MOV AX, DATA
0023	MOV DS, AX ;(2)
0024	
0025	:加法、FSW 实验
0026	MOV AX, DT3 ;(3)
0027	ADD AX, DT4 ;(4)
0028	MOV DT4, AX ;(5)
0029	
0030	MOV AL, DT1
	MOV BL, DT2

执行（2）之后的内部寄存器：

寄存器	地址	内容
EAX:	0x0000FE6	
ECX:	0x000E0000	
EDX:	0x00002002	
EBX:	0x0000FEA	
ESP:	0x00000C8	
EBP:	0x0000011E	
ESI:	0x00000100	
EDI:	0x0000024A	
EIP:	0x0000000D	
EFLAGS:	0x00007202	
CS:	0x0FF4	
SS:	0x0FE7	
DS:	0x0FE6	
ES:	0x0F7C	
FS:	0x0000	
GS:	0x0000	

地址	内容
0001	DATA SEGMENT
0002	DT1 DB 37
0003	DT2 DB 64H
0004	DT3 DW 0ABCDH
0005	DT4 DW 8765H
0006	DT5 EQU 0A7H
0007	DATA ENDS
0008	
0009	STKS SEGMENT STACK
0010	DW 100 DUP(0)
0011	STKS ENDS
0012	
0013	CODE SEGMENT
0014	ASSUME CS:CODE, DS:DATA, SS:STKS
0015	MAIN PROC FAR
0016	:初始化及数据段、堆栈段观察记录
0017	MOV AX, STKS ;(1)
0018	MOV SS, AX
0019	MOV SP, 100*2
0020	
0021	
0022	MOV AX, DATA
0023	MOV DS, AX ;(2)
0024	
0025	:加法、FSW 实验
0026	MOV AX, DT3 ;(3)
0027	ADD AX, DT4 ;(4)
0028	MOV DT4, AX ;(5)
0029	
0030	MOV AL, DT1
	MOV BL, DT2

数据段起始地址为 0FE6H 内容为 37、64H、0ABCDH、8765H，如下图所示：

0FE6:0000	25 64 CD AB 65 87 00 00	00 00 00 00 00 00 00 00
0FE6:0010	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
0FE6:0020	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
0FE6:0030	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
0FE6:0040	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
0FE6:0050	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
0FE6:0060	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
0FE6:0070	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00

堆栈段起始地址为 0FE7H 内容为:

0FE7:00C8	00 00 00 00 00 00 00 00	B8 E7 0F 8E D0 BC C8 00
0FE7:00D8	B8 E6 0F 8E D8 A1 02 00	03 06 04 00 A3 04 00 A0
0FE7:00E8	00 00 8A 1E 01 00 2A C3	A2 00 00 3B 06 02 00 77
0FE7:00F8	04 50 EB 05 90 FF 36 02	00 5A E8 04 00 B4 4C CD
0FE7:0108	21 B1 03 D3 C2 C3 E8 A7	FF 5B 0A C0 75 2C FF 76
0FE7:0118	FE FF 76 06 E8 D3 00 5B	5B 0B C0 75 0F FF 76 FE
0FE7:0128	FF 76 04 E8 C4 00 5B 5B	0B C0 74 04 80 4E FC 08
0FE7:0138	FF 76 FE E8 AC 09 5B 01	46 FE FF 76 FE FF 76 06

信息 命令 内存查看 栈查看 反汇编 变量

3.标号（3）执行完后 AX 的值为 0ABCDH, 标号（4）执行完后 AX 的值为 0332H, PSW 中的各项值为:

EAX:	0x00003332	0002	DT1 DB 37
ECX:	0x000E0000	0003	DT2 DB 64H
EDX:	0x00002002	0004	DT3 DW 0ABCDH
EBX:	0x00008FEA	0005	DT4 DW 8765H
ESP:	0x000000C8	0006	DT5 EQU 0A7H
EBP:	0x0000011E	0007	DATA ENDS
ESI:	0x00000100	0008	
EDI:	0x0000024A	0009	STKS SEGMENT STACK
EIP:	0x00000014	0010	DW 100 DUP(0)
EFLAGS:	0x00007213	0011	STKS ENDS
CS:	0x0FF4	0012	
SS:	0x0		IOPL = 11
DS:	0x0		NT = 1
ES:	0x0		RF = 0
FS:	0x0		VM = 0
GS:	0x0		AC = 0
	0x0		VIF = 0
	IF = 1		SF, AX
	DF = 0		IF, 100*2
	OF = 0		ID = 0
		0022	MOV AX, DATA
		0023	MOV DS, AX ; (2)

应为结果有进位所以 CF=1,应为结果底八位有奇数个 1 所以 PF=0, 因为底四位向高四维有进位, 所以 AF=1, 结果不为 0, 所以 ZF=0, 结果为正数, 所以 SF=0, 结果没有溢出所以 OF=0

标号（5）执行完后, PSW 各项值没有改变, DT4 的内容为 3332H

4.标号（6）执行前 AL=25H, BL=64H, 执行后 AL=0C1H, BL=64H

执行（6）之后的 PSW 中的值为：

CF	=	1	IOPL	=	11	D
PF	=	0	NT	=	1	A
AF	=	0	RF	=	0	
ZF	=	0	VM	=	0	
SF	=	1	AC	=	0	D
TF	=	0	VIF	=	0	D
IF	=	1	VIP	=	0	A
DF	=	0	ID	=	0	
OF	=	0				

标号（7）执行后 DT1 为 0C1H

5.标号（9）和（10）会执行（9）因为 $AX < DT3$ ，不满足跳转 NEXT1 的条件。

（9）执行后 $SP=SP-2$ ，SS 不改变，堆栈由：

0FE7:00C8	00 00 00 00 00 00 00 00	B8 E7 0F 8E D0 BC C8 00
0FE7:00D8	B8 E6 0F 8E D8 A1 02 00	03 06 04 00 A3 04 00 A0
0FE7:00E8	00 00 8A 1E 01 00 2A C3	A2 00 00 3B 06 02 00 77
0FE7:00F8	04 50 EB 05 90 FF 36 02	00 5A E8 04 00 B4 4C CD
0FE7:0108	21 B1 03 D3 C2 C3 E8 A7	FF 5B 0A C0 75 2C FF 76
0FE7:0118	FE FF 76 06 E8 D3 00 5B	5B 0B C0 75 0F FF 76 FE
0FE7:0128	FF 76 04 E8 C4 00 5B 5B	0B C0 74 04 80 4E FC 08
0FE7:0138	FF 76 FE E8 AC 09 5B 01	46 FE FF 76 FE FF 76 06

变为：

0FE7:00C6	C1 33 00 00 00 00 00 00	00 00 B8 E7 0F 8E D0 BC
0FE7:00D6	C8 00 B8 E6 0F 8E D8 A1	02 00 03 06 04 00 A3 04
0FE7:00E6	00 A0 00 00 8A 1E 01 00	2A C3 A2 00 00 3B 06 02
0FE7:00F6	00 77 04 50 EB 05 90 FF	36 02 00 5A E8 04 00 B4
0FE7:0106	4C CD 21 B1 03 D3 C2 C3	E8 A7 FF 5B 0A C0 75 2C
0FE7:0116	FF 76 FE FF 76 06 E8 D3	00 5B 5B 0B C0 75 0F FF
0FE7:0126	76 FE FF 76 04 E8 C4 00	5B 5B 0B C0 74 04 80 4E
0FE7:0136	FC 08 FF 76 FE E8 AC 09	5B 01 46 FE FF 76 FE FF

6.执行（12）后 $SP=SP-2$,SS 不改变，堆栈压入了一个 0035H

执行（14）后 $SP=SP+2$,SS 不改变，堆栈弹出一个 0035H

7.（13）处的指令 ROL 为循环左移指令，执行前 DX 为 33C1H，执行后 DX 为 9E09H

8. 堆栈从：

OFE7:00C8	00 00 00 00 00 00 00 00	B8 E7 0F 8E D0 BC C8 00
OFE7:00D8	B8 E6 0F 8E D8 A1 02 00	03 06 04 00 A3 04 00 A0
OFE7:00E8	00 00 8A 1E 01 00 2A C3	A2 00 00 3B 06 02 00 77
OFE7:00F8	04 50 EB 05 90 FF 36 02	00 5A E8 04 00 B4 4C CD
OFE7:0108	21 B1 03 D3 C2 C3 E8 A7	FF 5B 0A C0 75 2C FF 76
OFE7:0118	FE FF 76 06 E8 D3 00 5B	5B 0B C0 75 0F FF 76 FE
OFE7:0128	FF 76 04 E8 C4 00 5B 5B	0B C0 74 04 80 4E FC 08
OFE7:0138	FF 76 FE E8 AC 09 5B 01	46 FE FF 76 FE FF 76 06

变为:

OFE7:00C2	39 00 F4 0F 93 7A 00 00	00 00 00 00 00 00 B8 E7
OFE7:00D2	0F 8E D0 BC C8 00 B8 E6	0F 8E D8 A1 02 00 03 06
OFE7:00E2	04 00 A3 04 00 A0 00 00	8A 1E 01 00 2A C3 A2 00
OFE7:00F2	00 3B 06 02 00 77 04 50	EB 05 90 FF 36 02 00 5A
OFE7:0102	E8 04 00 B4 4C CD 21 B1	03 D3 C2 C3 E8 A7 FF 5B
OFE7:0112	0A C0 75 2C FF 76 FE FF	76 06 E8 D3 00 5B 5B 0B
OFE7:0122	C0 75 0F FF 76 FE FF 76	04 E8 C4 00 5B 5B 0B C0
OFE7:0132	74 04 80 4E FC 08 FF 76	FE E8 AC 09 5B 01 46 FE

9. 程序将跳转到 NEXT1

10. 课前预习 2:

11.1.

```

0001 DATA SEGMENT
0002     DT1 DB 1,2,'AB'
0003     COUNT EQU $-DT1
0004     DT2 DW 1,2,'AB'
0005     DT3 DW DT2
0006     DT4 DW $-DT2
0007     SN DB 8 DUP(?)
0008     NAM DB 30 DUP(?)
0009     DATA ENDS
0010
0011 CODE SEGMENT
0012     ASSUME CS:CODE,DS:DATA
0013 START:
0014     ;初始化及数据段观察记录
0015     MOV AX, DATA
0016     MOV DS, AX
    
```

12.

13. 2.

```

Elpin Systems at: (800) 723-9038 or www.elpin.com

Bochs BIOS, 1 cpu, $Revision: 1.103.2.2 $ $Date: 2004/02/02 22:39:22 $

ata0 master: Generic 1234 ATA-2 Hard-Disk (4 MBytes)
ata0 slave: Generic 1234 ATA-2 Hard-Disk (29 MBytes)

Booting from Hard Disk...
Starting MS-DOS...

C:\>set path=c:\dos;d:\tasm\bin

C:\>d:

D:\>cd d:\exec

D:\EXEC>deltree /Y .
Deleting .\|≤||3.exe...

D:\EXEC>..\all.com
loading file ... |≤||3.exe
load & exec
A_
  
```

14.

15.3.数据段内容:

0FE6:0000	01 02 41 42 01 00 02 00	42 41 04 00 08 00 00 00
0FE6:0010	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
0FE6:0020	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
0FE6:0030	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
0FE6:0040	B8 E6 0F 8E D8 B4 02 8A	16 00 00 CD 21 8A 16 02
0FE6:0050	00 CD 21 BB 00 00 B4 01	CD 21 3C 0D 75 02 74 07
0FE6:0060	88 87 0E 00 43 EB EF BA	16 00 B4 0A CD 21 8B CB
0FE6:0070	BB 00 00 B4 02 8A 97 0E	00 CD 21 43 49 75 F6 74

16.

17.DT1-DAT4 内存存储顺序和偏移地址:

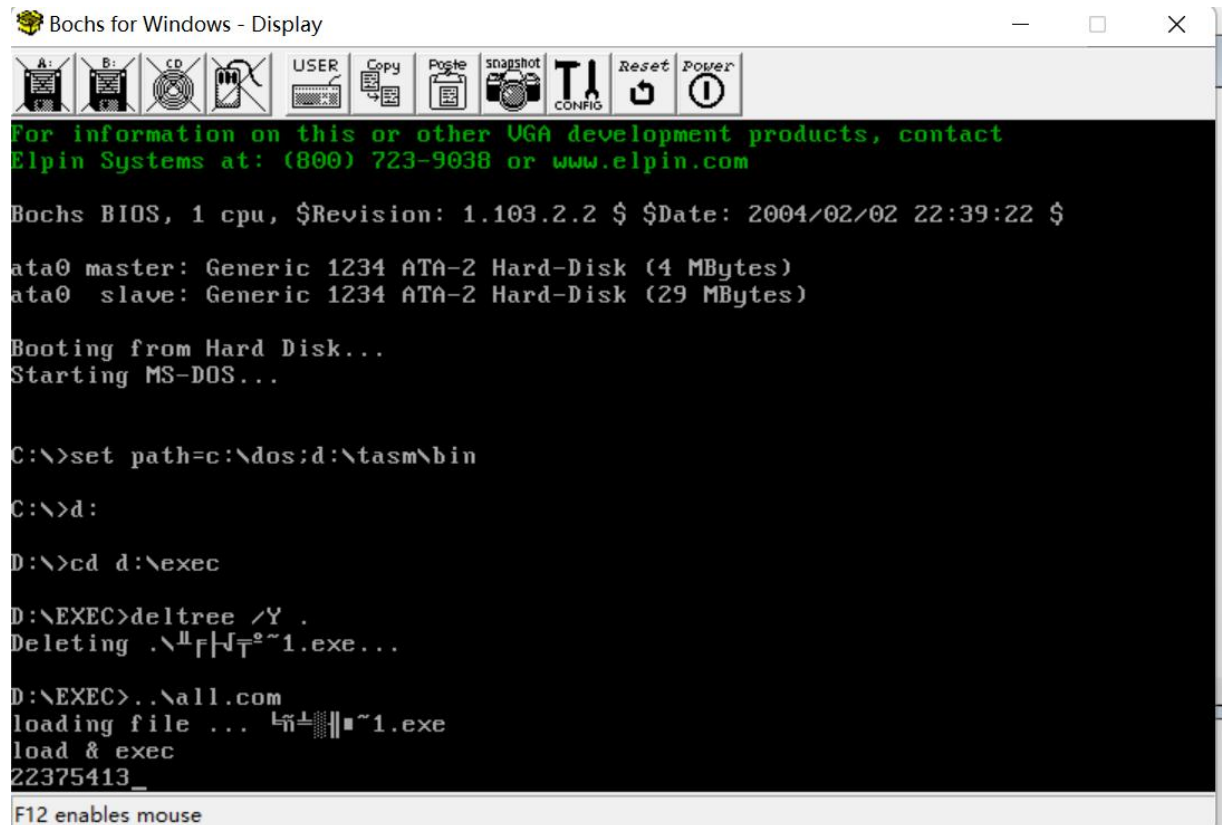
DT1:	<table><tr><td>1</td><td>0000H</td></tr><tr><td>2</td><td>0001H</td></tr><tr><td>A'</td><td>0002H</td></tr><tr><td>B'</td><td>0003H</td></tr></table>	1	0000H	2	0001H	A'	0002H	B'	0003H																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																							
1	0000H																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
2	0001H																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
A'	0002H																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
B'	0003H																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															

4. (1) 没有

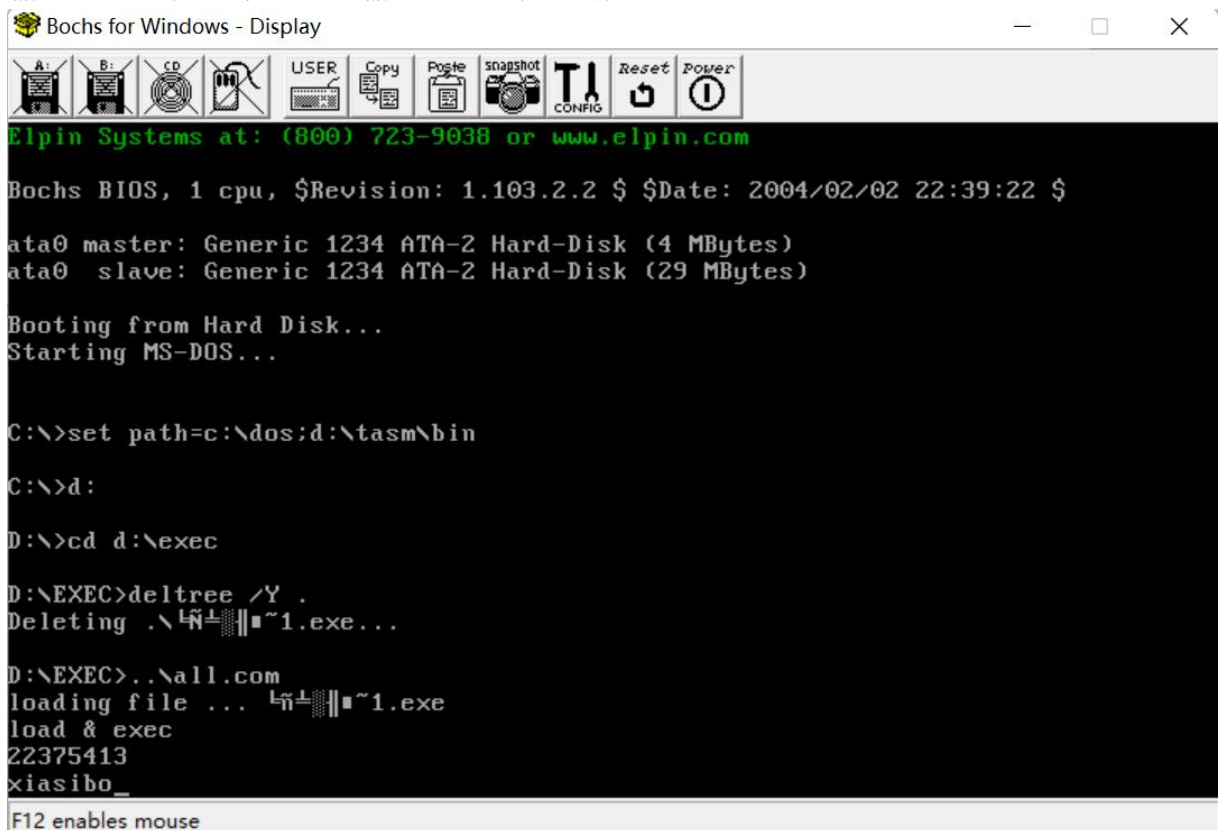
(2) DT1 定义的是字节, DT2 定义的是字, 以 1 来举例, DT1 定义的就是 01H, 而 DT2 定义的是 0001H, 占用两个字节, 以 'AB' 来说, DT1 是分为两个字节来定义, 而 DT2 是将看做一个整体一个字来定义, 'A' 是高字节, 'B' 是低字节

(3) DT4 的值为 8, 是从 DT2 到 DT4 定义的长度, 是用 DT4 的偏移地址减去 DT2 的偏移地址所得到的。

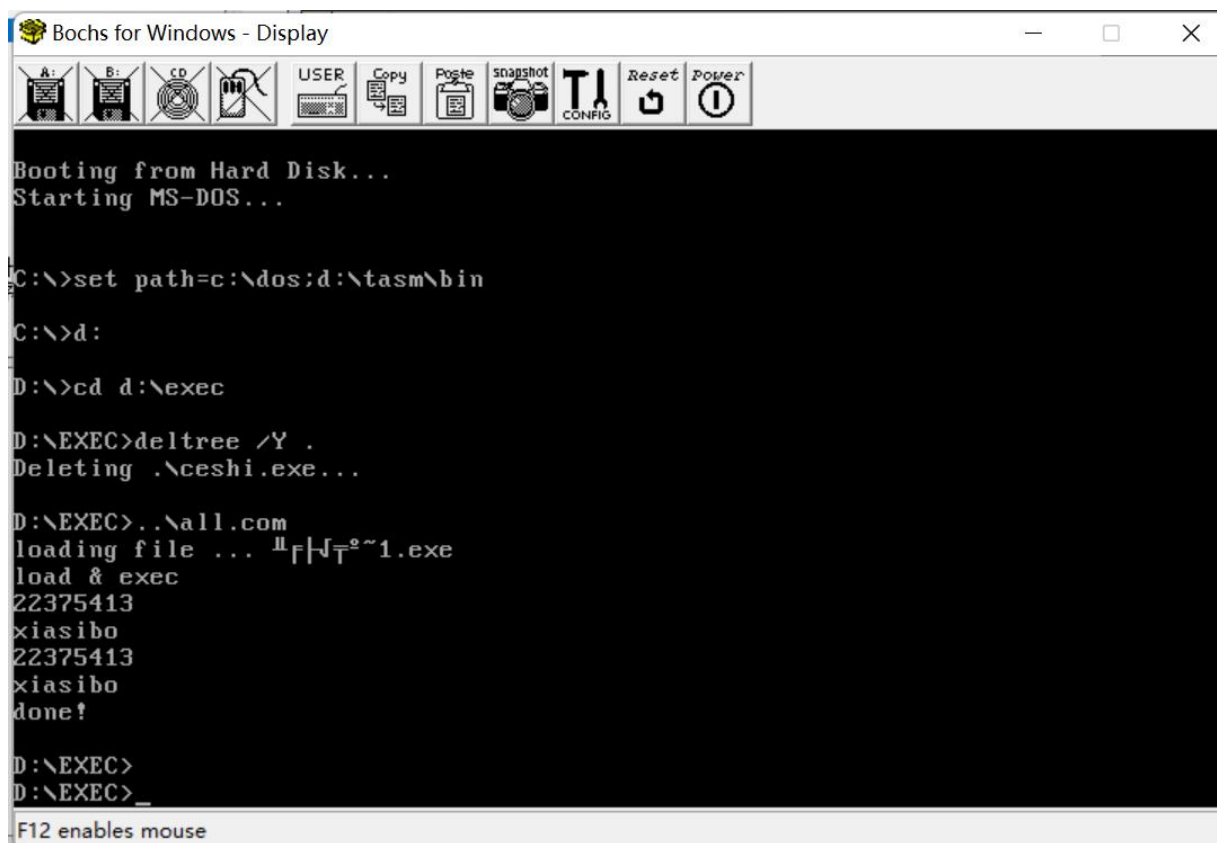
5.



输入学号后按回车键，再输入自己的姓名拼音，结果如下



6.



```
Bochs for Windows - Display
A: B: CD USER Copy Paste Snapshot CONFIG Reset Power
Booting from Hard Disk...
Starting MS-DOS...

C:\>set path=c:\dos;d:\tasm\bin
C:\>d:
D:\>cd d:\exec
D:\EXEC>deltree /Y .
Deleting .\ceshi.exe...

D:\EXEC>..\all.com
loading file ... 11f1j1~1.exe
load & exec
22375413
xiasibo
22375413
xiasibo
done!
D:\EXEC>
D:\EXEC>_
F12 enables mouse
```